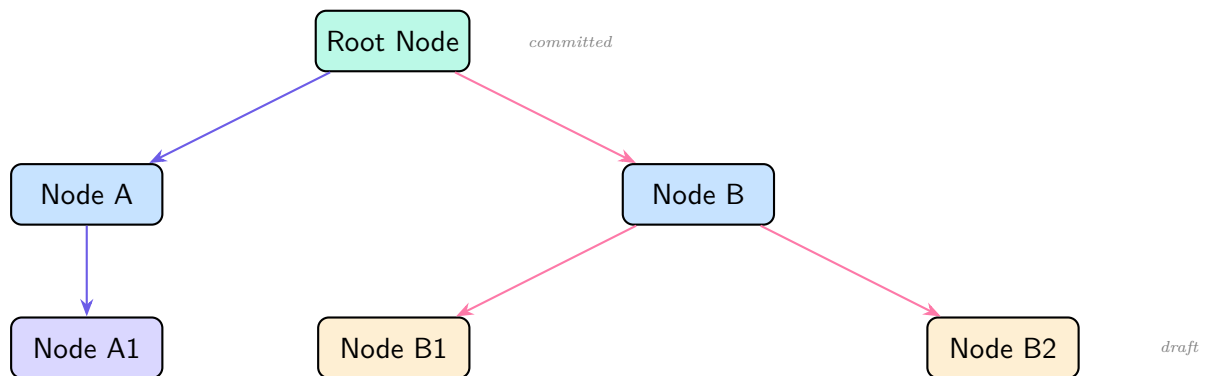


Zygote

Architecture & Codebase Guide



A complete guide to understanding every file, data structure, and data flow in the Zygote VS Code extension.

Version 0.1.0 | May 29, 2026

Total codebase: ~4,900 lines of TypeScript

This document is designed to be read section by section. Each section is self-contained and builds on the previous ones.

Contents

1	What Is Zygote?	2
1.1	The Problem Zygote Solves	2
1.2	The Mental Model	2
1.3	Technology Stack	3
2	Project Directory Structure	4
2.1	Top-Level Layout	4
2.2	Complete File Tree	4
3	The Three-Layer Architecture	6
3.1	Architecture Diagram	6
3.2	Layer Responsibilities	6
4	Core Data Model	7
4.1	ZygoteTree — The Root Container	7
4.2	ZygoteNode — The Atomic Unit of Work	7
4.3	Node Status Lifecycle	8
4.4	ZygoteBranch — A Named Line of Work	8
4.5	How Tree Structure Works	8
5	Message Protocol	10
5.1	Webview → Extension (User Actions)	10
5.2	Extension → Webview (State Updates)	10
6	Module Deep Dives	11
6.1	Entry Point: extension.ts	11
6.2	The Brain: ZygotePanel.ts	11
6.3	Pure State Logic: tree.ts	12
6.4	Disk I/O: persistence.ts	13
6.5	File Versioning: snapshots.ts	13
6.6	API Client: claudes.ts	14
6.7	Agent Tools: tools.ts	14
6.8	Agent Execution: runner.ts	15
6.8.1	The runPreview() Workflow	15
6.8.2	Context Construction	15
6.8.3	Two Backend Implementations	16
7	The React UI	17
7.1	UI Layout	17
7.2	Component Breakdown	17
7.3	Key State in App.tsx	18

8	Complete Data Flow Walkthroughs	19
8.1	Walkthrough 1: Creating a Node and Running the Agent	19
8.2	Walkthrough 2: Forking a Branch	20
8.3	Walkthrough 3: Rejecting a Preview	21
8.4	Walkthrough 4: Switching Branches	21
9	File Snapshot System (Deep Dive)	22
9.1	How It Works	22
9.2	Deduplication Example	22
10	Persistence Format	23
11	Debug System	24
12	Build and Run	25
12.1	Build Scripts	25
12.2	Development Workflow	25
12.3	Extension Manifest (package.json)	25
13	What's NOT Built Yet (v0.1 Scope)	26
14	Key Design Decisions Summary	27
15	Quick Reference: File → Purpose Lookup	28

1 What Is Zygote?

The One-Sentence Summary

Zygote is a VS Code extension that replaces linear AI chat with a **tree of work nodes**, where each node is a committed unit of work with its own file snapshot, and you can fork branches to try alternatives.

1.1 The Problem Zygote Solves

Traditional AI coding assistants (ChatGPT, Copilot Chat, Claude chat) work in a **linear conversation**. You ask something, the AI responds, and you continue forward. If the AI goes in a wrong direction, you either:

- (a) Undo manually and re-prompt (losing the context of what was tried)
- (b) Start a new chat (losing all prior context)
- (c) Keep going and hope to fix it later

None of these are good. Zygote solves this by making the conversation a **tree**, not a line.

1.2 The Mental Model

Think of Zygote like **Git for AI conversations**:

Node = A commit. Contains a prompt, the AI's response, and a snapshot of all files at that point.

Branch = A named line of work. Has a head node (the tip).

Fork = Creating a new branch from any committed node. Files revert to that node's state.

Switch = Changing which branch is active. Files automatically restore.

Key insight: Every committed node stores the exact state of every file in your workspace at that moment. When you switch branches or fork, Zygote restores those files. You can freely explore alternatives without fear of losing work.

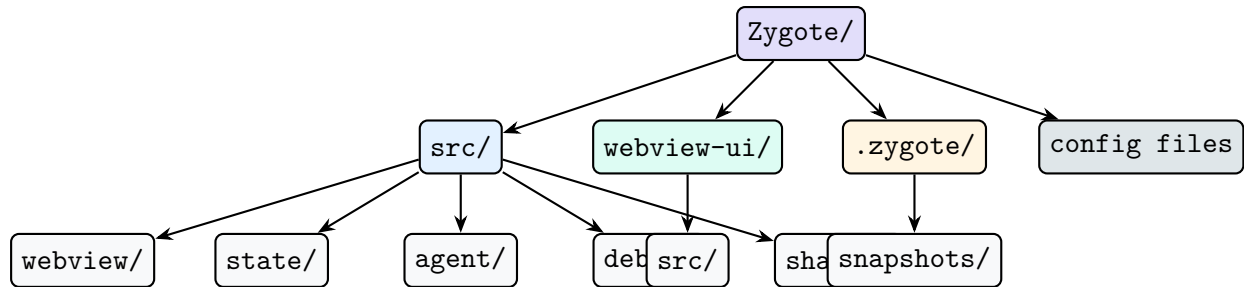
1.3 Technology Stack

Layer	Technology	Purpose
Extension Host	Node.js, TypeScript	Business logic, state, agent
VS Code API	<code>vscode</code> module	Extension lifecycle, webview, file I/O
Webview UI	React 19, Vite 6	UI rendering in webview sandbox
Markdown	react-markdown, rehype-highlight	Render agent output
Agent (SDK)	@anthropic-ai/sdk v0.98+	Claude API calls
Agent (CLI)	claude-agent-sdk v0.3+	CLI backend fallback
Build	TypeScript 5.8, Vite 6	Compilation and bundling

2 Project Directory Structure

This section maps every directory and file in the project. Use it as a reference when navigating the codebase.

2.1 Top-Level Layout



2.2 Complete File Tree

Zygote/ -- Root Directory

package.json	VS Code extension manifest, dependencies, scripts
tsconfig.json	TypeScript config (ES2022, strict, Node16 modules)
SPEC.md	Product specification (693 lines)
PLAN.md	Development plan and checklist
zygote-strategic-roadmap.md	18–24 month strategic roadmap

src/ -- Extension Host Source (Node.js)

extension.ts	68 lines	Entry point. Registers VS Code commands.
webview/ZygotePanel.ts	~900 lines	The brain. Message routing, state orchestration.
webview/html.ts		HTML scaffold for webview with CSP headers.
state/tree.ts	~450 lines	Pure state operations on the tree.
state/persistence.ts	90 lines	Load/save tree.json to disk.
state/snapshots.ts	107 lines	Content-addressable file storage (SHA-256).
agent/claude.ts	84 lines	Anthropic API client singleton.
agent/tools.ts	192 lines	4 tool definitions + executors.
agent/runner.ts	~450 lines	Agent execution loop, preview workflow.
debug/logger.ts	317 lines	Debug logging + tree dumps.
shared/types.ts	84 lines	Shared TypeScript type definitions.

webview-ui/src/ -- React UI (Webview Sandbox)

<code>main.tsx</code>		React bootstrap entry point.
<code>App.tsx</code>	<i>~470 lines</i>	Main app component, layout, state.
<code>vscode-api.ts</code>		Wrapper around <code>acquireVsCodeApi()</code> .
<code>components/TreeCanvas.tsx</code>		Interactive tree visualization.
<code>components/NodeDetail.tsx</code>		Right panel: selected node details.
<code>components/NodeChat.tsx</code>		Chat interface within a node.
<code>components/FloatingChat.tsx</code>		Ephemeral quick-ask popup.
<code>components/BranchSelector.tsx</code>		Branch switching dropdown.
<code>components/MarkdownRenderer.tsx</code>		Markdown rendering with syntax highlighting.
<code>components/ThinkingBlock.tsx</code>		Expandable thinking blocks display.
<code>components/Tree.tsx</code>		Tree display (may be superseded).

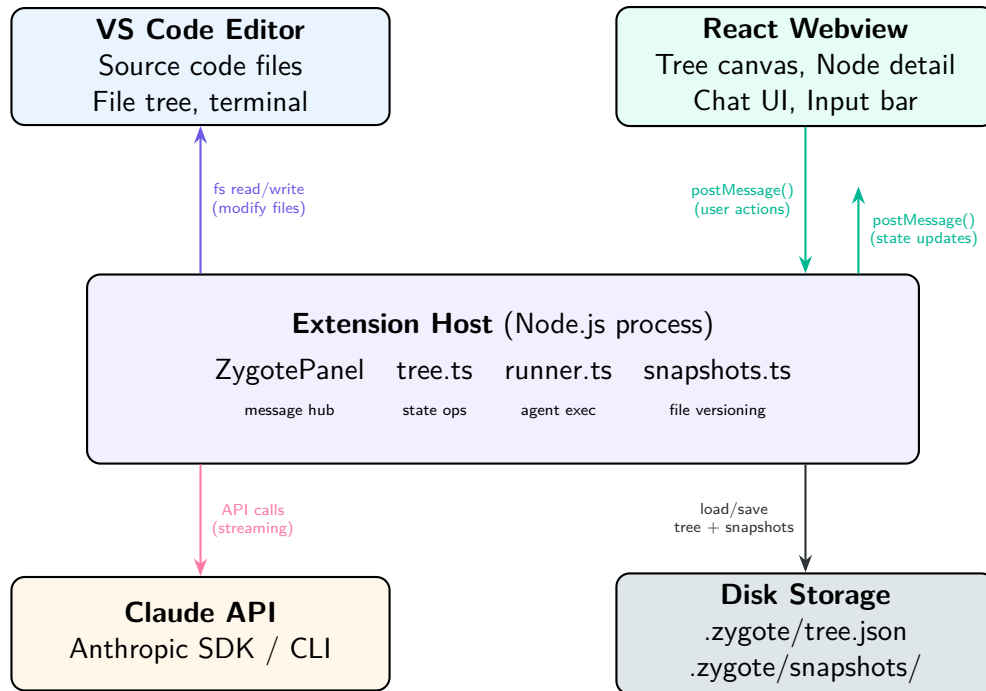
.zygote/ -- Runtime Data (per workspace, gitignored)

<code>tree.json</code>	Full tree state (nodes, branches, snapshots).
<code>snapshots/<sha256></code>	Content-addressable file blobs.

3 The Three-Layer Architecture

Zygote has three distinct layers. Understanding how they communicate is the key to understanding the whole system.

3.1 Architecture Diagram



3.2 Layer Responsibilities

Layer	Runs In	Responsibility
VS Code Editor	VS Code main process	Displays source code files. Agent modifies files here via <code>vscode.workspace.fs</code> .
React Webview	Sandboxed iframe	UI only. Renders the tree, node details, chat. Cannot access the filesystem.
Extension Host	Node.js child process	Source of truth. Holds the tree, runs the agent, does file I/O, persists state.

Critical rule: The Extension Host is the single source of truth. The webview is a *view layer only*. It receives the full tree via `treeUpdated` messages and renders it. All mutations happen in the extension host.

4 Core Data Model

Everything in Zygote revolves around three data structures: **ZygoteTree**, **ZygoteNode**, and **ZygoteBranch**. They are defined in `src/shared/types.ts`.

4.1 ZygoteTree — The Root Container

```
1 interface ZygoteTree {
2   workspaceRoot: string;           // VS Code workspace path
3   rootNodeIds: NodeId[];          // Top-level entry points
4   nodes: Record<NodeId, ZygoteNode>; // FLAT map of all nodes
5   branches: Record<BranchId, ZygoteBranch>;
6   activeBranchId: BranchId;        // Currently "checked out"
                                     branch
7 }
```

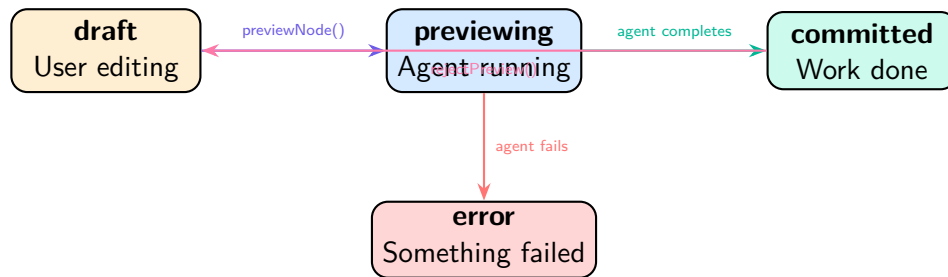
Why a flat map? Nodes are stored in a flat `Record<NodeId, ZygoteNode>`, not nested objects. The tree structure comes from each node's `parentId` pointer. This gives $O(1)$ lookup by ID, easy serialization to JSON, and simple traversal.

4.2 ZygoteNode — The Atomic Unit of Work

Each node represents one prompt–response cycle with its associated file state.

```
1 interface ZygoteNode {
2   id: NodeId;                      // UUID
3   parentId: NodeId | null;         // Tree link (null = root)
4   branchId: BranchId;              // Which branch owns this
                                     node
5   status: NodeStatus;              //
                                     'draft'/'previewing'/'committed'/'error'
6   title: string;                   // Display label
                                     (auto-summarized)
7   prompt: string;                  // User's instruction
8   chat: ChatMessage[];             // Pre-execution conversation
9   agentResponse?: string;          // Claude's full text output
10  thinkingContent?: string;         // Extended thinking blocks
11  toolCalls?: ToolCall[];           // File operations performed
12  workspaceSnapshot?: {            // File state at commit time
13    fileHashes: Record<string, FileHash>
14  };
15  tokens?: { input: number; output: number };
16  error?: string;                   // Error message if
                                     status='error'
17 }
```

4.3 Node Status Lifecycle



- **draft**: Node exists but agent hasn't run. User can edit prompt, add chat messages.
- **previewing**: Agent is currently executing. Workspace is locked.
- **committed**: Agent finished. Results (response, tool calls, file snapshot) are stored.
- **error**: Agent execution failed. Error message stored in node.

4.4 ZygoteBranch — A Named Line of Work

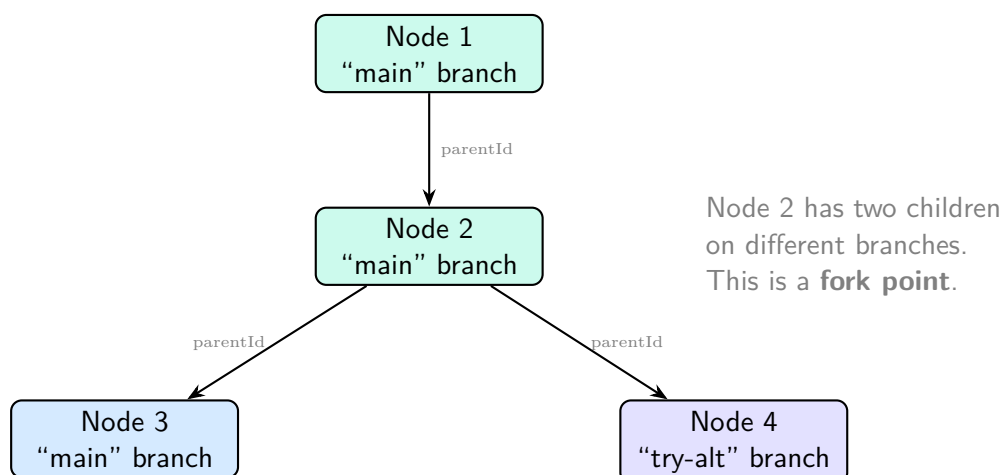
```

1 interface ZygoteBranch {
2   id: BranchId; // UUID
3   name: string; // "main", "try-middleware", etc.
4   headNodeId: NodeId | null; // Tip of this branch
5   parentBranchId?: BranchId; // Forked from which branch
6   forkedAtNodeId?: NodeId; // Forked at which node
7   createdAt: number; // Timestamp
8 }

```

Forking is O(1). When you fork, Zygote only creates a new branch entry pointing to the fork node. No nodes are copied. The new branch simply shares ancestor nodes with the original branch.

4.5 How Tree Structure Works



The nodes are stored flat:

```
1 nodes: {  
2   "node-1": { id: "node-1", parentId: null,      branchId:  
3     "main-branch" },  
4   "node-2": { id: "node-2", parentId: "node-1", branchId:  
5     "main-branch" },  
6   "node-3": { id: "node-3", parentId: "node-2", branchId:  
7     "main-branch" },  
8   "node-4": { id: "node-4", parentId: "node-2", branchId:  
9     "try-alt" },  
10 }
```

5 Message Protocol

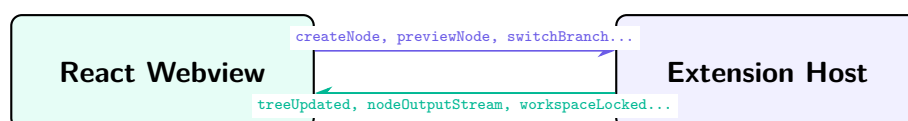
The webview and extension host communicate via `postMessage()`. Every user action and every state update flows through this channel.

5.1 Webview → Extension (User Actions)

Message Type	Payload	What Happens
<code>createNode</code>	<code>parentId</code> , <code>prompt</code>	Creates draft node, auto-runs preview
<code>updateNodePrompt</code>	<code>nodeId</code> , <code>prompt</code>	Edits draft node's prompt
<code>updateNodeTitle</code>	<code>nodeId</code> , <code>title</code>	Renames node
<code>appendChatMessage</code>	<code>nodeId</code> , <code>role</code> , <code>content</code>	Adds chat message to node
<code>requestChatResponse</code>	<code>nodeId</code>	Runs agent on node's chat
<code>previewNode</code>	<code>nodeId</code>	Begins preview execution
<code>rejectPreview</code>	<code>nodeId</code>	Reverts files, returns to draft
<code>switchBranch</code>	<code>branchId</code>	Switches active branch + restores files
<code>deleteNode</code>	<code>nodeId</code>	Deletes leaf draft node
<code>deleteBranch</code>	<code>branchId</code>	Cascade deletes branch + children
<code>saveAndCheckout</code>	<code>fromNodeId</code> , <code>toNodeId</code>	Checkpoint and restore
<code>quickAsk</code>	<code>prompt</code>	Ephemeral floating chat (not persisted)

5.2 Extension → Webview (State Updates)

Message Type	Payload	What It Does
<code>treeUpdated</code>	tree (full ZygoteTree)	Full state refresh
<code>nodeOutputStream</code>	<code>nodeId</code> , <code>delta</code>	Streaming agent text chunk
<code>nodeThinkingStream</code>	<code>nodeId</code> , <code>delta</code>	Streaming thinking block
<code>workspaceLocked</code>	<code>nodeId</code> , <code>title</code>	Indicates agent is running
<code>workspaceUnlocked</code>	(none)	Agent finished
<code>error</code>	message, <code>nodeId</code> ?	Error notification
<code>quickAskResponse</code>	response	Floating chat answer



6 Module Deep Dives

This section walks through each source file in detail.

6.1 Entry Point: `extension.ts`

`src/extension.ts` (68 lines)

This is where VS Code loads Zygote. It runs once when the extension activates.

What it does:

1. Initializes the debug logger
2. Registers three VS Code commands:
 - `zygote.open` — loads the tree from disk, creates/shows the webview panel
 - `zygote.captureSnapshot` — saves a debug snapshot
 - `zygote.dumpTree` — dumps the full tree for debugging

Activation trigger: The `package.json` declares `onCommand:zygote.open`, so VS Code only loads Zygote when the user first runs that command.

6.2 The Brain: `ZygotePanel.ts`

`src/webview/ZygotePanel.ts` (~900 lines)

This is the largest and most important file. It is a **singleton class** that:

1. Creates and manages the VS Code webview panel
2. Holds the `ZygoteTree` (the single source of truth)
3. Routes every message from the webview to the right handler
4. Orchestrates complex workflows (create node → run agent → update tree)
5. Handles file restoration during branch switches

Key instance fields:

- `tree`: `ZygoteTree` — the current tree state
- `activeRunNodeId`: `NodeId | null` — tracks which node is currently being previewed
- `pendingCheckoutNodeId`: `NodeId | null` — file restore queued for after execution

Message routing: Every message from the webview hits `handleMessage()`, which dispatches to specific handlers like `handleCreateNode()`, `handlePreviewNode()`, `handleSwitchBranch()`, etc.

6.3 Pure State Logic: `tree.ts`

`src/state/tree.ts` (~450 lines)

This file contains **pure functions** that operate on the `ZygoteTree` data structure. No side effects, no I/O, no VS Code API calls. Each function takes a tree, returns a modified tree.

Functions:

Function	What It Does
<code>createDefaultTree()</code>	Creates empty tree with a “main” branch
<code>createNode(tree, parentId, prompt)</code>	Creates a draft node. If parent already has a child on the active branch, auto-forks a new branch
<code>updateNodePrompt()</code>	Edits a draft node’s prompt (immutable copy)
<code>updateNodeTitle()</code>	Renames a node
<code>setNodeStatus()</code>	Transitions node state with optional extra data
<code>appendChatMessage()</code>	Adds a message to a node’s chat array
<code>deleteNode()</code>	Removes a leaf-only node
<code>forkNode()</code>	Creates new branch at a node ($O(1)$)
<code>deleteBranch()</code>	Cascade deletes branch + all descendant nodes
<code>switchBranch()</code>	Changes <code>activeBranchId</code>
<code>rejectPreview()</code>	Reverts a committed node back to draft
<code>getBranchNodes()</code>	Returns the path from root to branch head
<code>getNodeChildren()</code>	Returns all children of a node

Auto-Branching Algorithm

When `createNode()` is called with a parent that already has a child on the active branch:

```

1 // Pseudocode
2 if (parent already has child on active branch) {
3     // Auto-fork: create new branch
4     newBranch = { parentBranchId: activeBranch,
5                   forkedAtNodeId: parentId }
6     // New node goes on the new branch
7 }

```

Why? This prevents accidentally creating a linear chain when the user wants to try an alternative. If you're at Node 2 and it already has a child (Node 3) on the current branch, creating another child automatically forks into a new branch.

6.4 Disk I/O: persistence.ts

`src/state/persistence.ts` (90 lines)

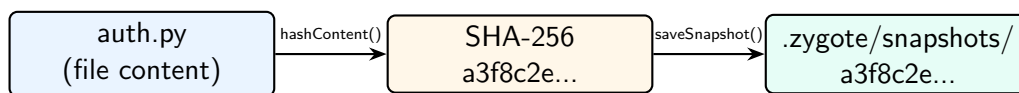
Simple file I/O for the tree state:

- `loadTree(workspaceRoot)` — reads `.zygote/tree.json`. If missing, returns a fresh default tree. Includes recovery logic: fixes orphaned nodes, resets nodes stuck in “previewing” status.
- `saveTree(workspaceRoot, tree)` — writes tree as JSON to `.zygote/tree.json`. Auto-creates the `.zygote/` directory.
- `ensureZygoteDir(workspaceRoot)` — ensures `.zygote/snapshots/` exists.

6.5 File Versioning: snapshots.ts

`src/state/snapshots.ts` (107 lines)

Content-addressable file storage, like a mini-Git object store.



Key functions:

- `hashContent(content)` — SHA-256 hash of file content
- `saveSnapshot(workspaceRoot, content)` — Store blob under its hash. Skip if already exists (dedup).
- `loadSnapshot(workspaceRoot, hash)` — Retrieve stored content by hash.
- `captureWorkspaceSnapshot(root, paths)` — Hash multiple files at once.

- `restoreWorkspaceSnapshot(root, fileHashes)` — Restore all files from their hashes.

Deduplication: If two nodes have the same version of a file, they share the same snapshot blob. Storage is proportional to the number of *unique* file contents, not the number of nodes.

6.6 API Client: `claude.ts`

`src/agent/claude.ts` (84 lines)

Singleton wrapper around the Anthropic SDK:

- `getClient(secretStorage)` — Creates or returns the cached Anthropic client. Reads API key from VS Code's secret storage or the `ANTHROPIC_API_KEY` environment variable.
- `resetClient()` — Clears the cached client (e.g., after API key change).
- `sendMessage(client, options)` — Calls the Claude API with streaming enabled.
- Model used: `claude-sonnet-4-20250514`

6.7 Agent Tools: `tools.ts`

`src/agent/tools.ts` (192 lines)

Defines the 4 tools that Claude can use during execution:

Tool	Parameters	Behavior
<code>read_file</code>	path (relative)	Reads file content from workspace
<code>list_files</code>	path (relative)	Lists directory contents. Filters out <code>.zygote/</code> , <code>node_modules</code>
<code>write_file</code>	path, content	Writes or creates a file. Auto-creates parent directories
<code>search_files</code>	pattern (regex)	Regex search across all workspace files. Max 50 matches

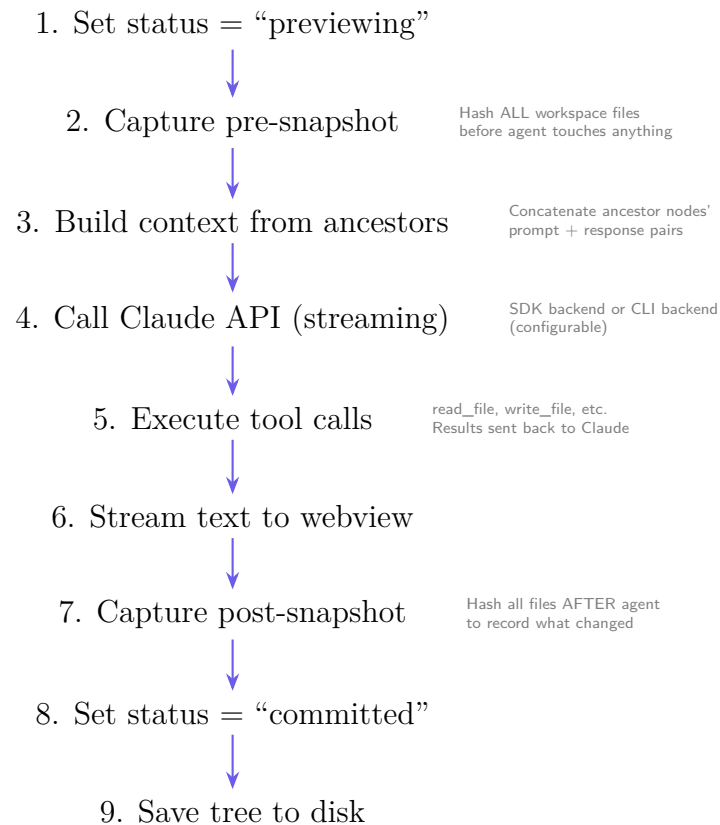
`executeTool(workspaceRoot, toolName, input)` dispatches a tool call to the right handler and returns the result.

6.8 Agent Execution: runner.ts

`src/agent/runner.ts` (~450 lines)

This is where the AI actually runs. It manages the full lifecycle of a preview.

6.8.1 The runPreview() Workflow



6.8.2 Context Construction

When the agent runs, it needs to know about prior work. The context is built by walking up the ancestor chain:

```

1 // Pseudocode for context building
2 ancestors = walkUp(node.parentId) // get all ancestors in order
3 context = ancestors.map(n =>
4   'User: ${n.prompt}\nAssistant: ${n.agentResponse}'
5 ).join("\n\n---\n\n")
6
7 // Plus the current node's chat messages
8 context += node.chat.map(m =>
9   `${m.role}: ${m.content}`
10 ).join("\n\n")
  
```

This means the agent always sees the full decision history leading to the current node.

6.8.3 Two Backend Implementations

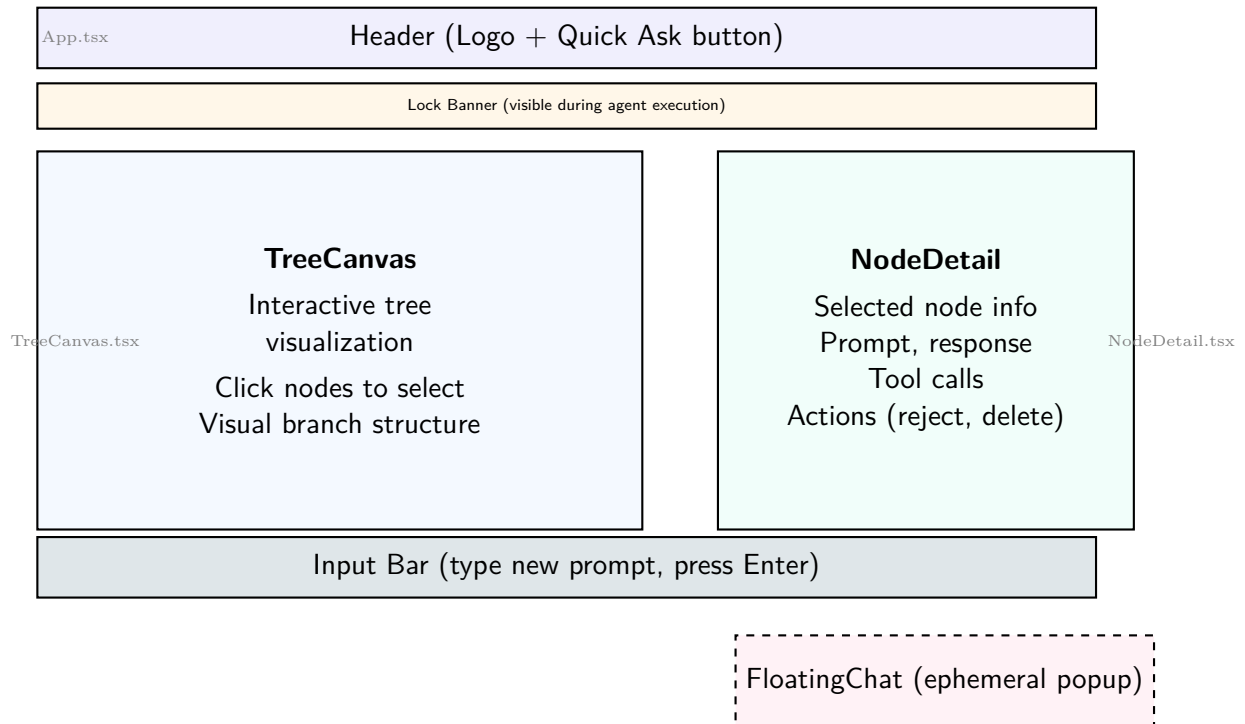
Backend	How It Works	When Used
SDK	Direct API calls via <code>@anthropic-ai/sdk</code>	When API key is available
CLI	Spawns <code>claude -print -output-format stream-json</code> as child process	Fallback when SDK unavailable

`resolveBackend()` checks for an SDK API key first, falls back to CLI.

7 The React UI

The webview is a React app built with Vite, running inside a VS Code webview sandbox.

7.1 UI Layout



7.2 Component Breakdown

Component	Responsibility
App.tsx (470 lines)	Root component. Manages all state: tree, selected node, streaming text, workspace lock, error messages. Renders layout.
TreeCanvas.tsx	Renders the tree as an interactive canvas/SVG. Nodes connected with lines. Click to select.
NodeDetail.tsx	Right panel. Shows prompt, agent response (markdown), thinking blocks, tool calls, action buttons.
NodeChat.tsx	Chat interface within a draft node for pre-execution conversation.
FloatingChat.tsx	Ephemeral quick-ask dialog. Not persisted to tree.
BranchSelector.tsx	Dropdown to switch between branches.
MarkdownRenderer.tsx	Renders agent output using <code>react-markdown</code> + syntax highlighting.
ThinkingBlock.tsx	Expandable/collapsible display of Claude's thinking blocks.

7.3 Key State in App.tsx

```
1 // Core state variables in App.tsx
2 const [tree, setTree] = useState<ZygoteTree | null>(null);
3 const [selectedNodeId, setSelectedNodeId] = useState<NodeId |
  null>(null);
4 const [streamingText, setStreamingText] =
  useState<Record<NodeId, string>>({});
5 const [thinkingText, setThinkingText] =
  useState<Record<NodeId, string>>({});
6 const [workspaceLock, setWorkspaceLock] = useState<{nodeId,
  title} | null>(null);
7 const [errorMessage, setErrorMessage] = useState<string |
  null>(null);
8 const [inputValue, setInputValue] = useState<string>("");
```

8 Complete Data Flow Walkthroughs

These walkthroughs trace exactly what happens for each major user action, from click to result.

8.1 Walkthrough 1: Creating a Node and Running the Agent

Step 1: User types “refactor auth.py” and presses Enter

- `App.tsx` calls `postMessage({ type: 'createNode', parentId: null, prompt: 'refactor auth.py' })`

Step 2: Extension receives message

- `ZygotePanel.handleMessage()` routes to `handleCreateNode()`
- Calls `createNode(tree, null, 'refactor auth.py')` from `tree.ts`
- New draft node created with UUID, added to `tree.nodes`
- `tree.rootNodeIds` updated (since `parentId` is null)
- Branch head updated to point to new node

Step 3: Auto-summarize title (non-blocking)

- Calls `handleQuickAsk('Summarize as 3-5 word title: refactor auth.py')`
- Updates `node.title` when response arrives
- Sends `treeUpdated` to webview

Step 4: Immediately starts preview execution

- Sends `workspaceLocked { nodeId, title }` to webview
- Calls `runPreview(tree, nodeId, secretStorage, callbacks)`

Step 5: Inside `runPreview()`:

- Sets node status to “previewing”
- `captureWorkspaceSnapshot()` — hashes ALL workspace files (pre-snapshot)
- Builds context: since this is a root node, context is just the prompt
- Calls Claude API with the 4 tool definitions
- **Streaming loop:** For each chunk from Claude:
 - Text → sends `nodeOutputStream { nodeId, delta }` to webview
 - Thinking → sends `nodeThinkingStream { nodeId, delta }` to webview
 - Tool call → executes via `executeTool()`, sends result back to Claude

- After Claude finishes: `captureWorkspaceSnapshot()` again (post-snapshot)
- Sets node status to “committed” with `agentResponse`, `toolCalls`, `workspaceSnapshot`
- `saveTree()` writes to disk

Step 6: Webview receives updates

- `nodeOutputStream` → accumulates in `streamingText[nodeId]`
- `treeUpdated` → replaces full tree state
- Node in `TreeCanvas` shows “committed” status with green indicator
- `NodeDetail` shows rendered markdown response + tool calls

8.2 Walkthrough 2: Forking a Branch

Step 1: User clicks “Fork” on a committed node

- Webview sends `{ type: 'forkNode', nodeId: 'abc123', newBranchName: 'try-simpler' }`

Step 2: Extension creates fork

- Calls `forkNode(tree, 'abc123', 'try-simpler')` from `tree.ts`
- Creates new `ZygoteBranch`: `{ parentBranchId, forkedAtNodeId: 'abc123' }`
- This is $O(1)$ — no nodes are copied

Step 3: File state restoration

- Reads `tree.nodes['abc123'].workspaceSnapshot.fileHashes`
- Calls `restoreWorkspaceSnapshot(root, fileHashes)`
- For each file hash: reads blob from `.zygote/snapshots/`, writes to workspace
- VS Code editor now shows files as they were when Node `abc123` was committed

Step 4: Branch switch

- `tree.activeBranchId` set to new branch ID
- `saveTree()` persists
- Sends `treeUpdated` to webview

Step 5: Webview updates

- `TreeCanvas` shows new branch line
- `BranchSelector` shows “try-simpler” as active
- User can now type a new prompt for the new branch

8.3 Walkthrough 3: Rejecting a Preview

Step 1: User clicks “Reject” on a committed node

- Webview sends { type: 'rejectPreview', nodeId }

Step 2: Extension reverts

- Calls `rejectPreview(tree, nodeId)` from `tree.ts`
- Sets node status back to “draft”
- Clears `agentResponse`, `toolCalls`, `workspaceSnapshot`
- Restores files from the *pre-snapshot* (the state before the agent ran)

Step 3: User can now edit the prompt and try again

- Node is back in draft state
- Files are back to their pre-execution state
- Full preview–reject–edit–preview loop is possible

8.4 Walkthrough 4: Switching Branches

Step 1: User selects a different branch from BranchSelector

- Webview sends { type: 'switchBranch', branchId }

Step 2: Extension switches

- `switchBranch(tree, branchId)` sets `activeBranchId`
- Finds the head node of the target branch
- Restores files from head node's `workspaceSnapshot.fileHashes`
- Your editor now shows the files as they were on that branch

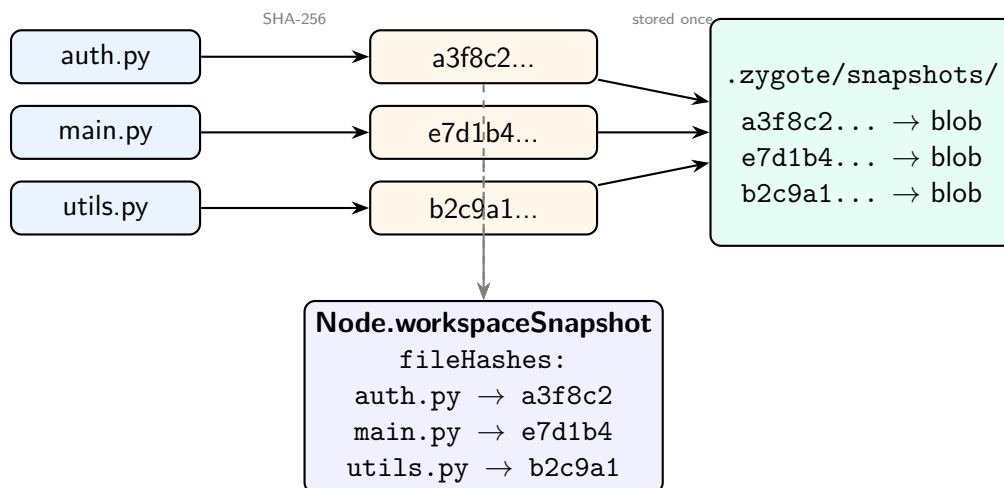
Step 3: Webview re-renders

- `TreeCanvas` highlights the active branch
- `NodeDetail` updates to show the head node

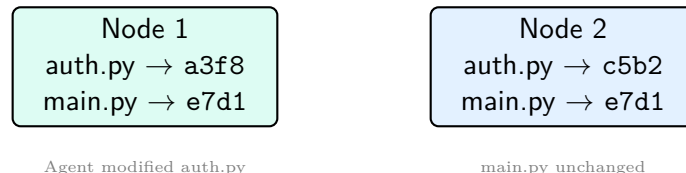
9 File Snapshot System (Deep Dive)

The snapshot system is how Zygote guarantees that switching branches or rejecting previews always restores the exact file state. It works like Git's object store.

9.1 How It Works



9.2 Deduplication Example



e7d1 blob stored only once!
Both nodes point to the same hash.

10 Persistence Format

All state is stored in `.zygote/tree.json`. Here is an annotated example:

```
1 {
2   "workspaceRoot": "/path/to/workspace",
3   "activeBranchId": "branch-main-uuid",
4   "rootNodeIds": ["node-1-uuid"],
5
6   "nodes": {
7     "node-1-uuid": {
8       "id": "node-1-uuid",
9       "parentId": null,
10      "branchId": "branch-main-uuid",
11      "status": "committed",
12      "title": "Refactor auth module",
13      "prompt": "Refactor the authentication...",
14      "chat": [
15        { "role": "user", "content": "...", "timestamp":
16          1234567890 }
17      ],
18      "agentResponse": "I'll refactor the auth module...",
19      "toolCalls": [
20        {
21          "tool": "read_file",
22          "input": { "path": "auth.py" },
23          "result": { "ok": true, "data": "..." }
24        }
25      ],
26      "workspaceSnapshot": {
27        "fileHashes": {
28          "auth.py": "a3f8c2e...",
29          "main.py": "e7d1b4f..."
30        }
31      },
32      "tokens": { "input": 500, "output": 800 }
33    }
34  },
35
36  "branches": {
37    "branch-main-uuid": {
38      "id": "branch-main-uuid",
39      "name": "main",
40      "headNodeId": "node-1-uuid",
41      "createdAt": 1234567890
42    }
43  }
```

11 Debug System

`src/debug/logger.ts` (317 lines)

The `DebugLogger` class writes structured logs to `.debug/session.jsonl` for diagnosing issues.

Log methods:

- `info(category, message, data?)` — informational
- `warn(category, message, data?)` — warnings
- `error(category, message, data?)` — errors

Diagnostic methods:

- `captureSnapshot(tree, extra)` — saves a full error analysis to `.debug/snapshot.json`
- `dumpTree(tree, extra)` — writes a human-readable markdown dump to `.debug/tree-dump.md` including:
 - Branch summary (name, head, parent)
 - Full node hierarchy (indented)
 - Status icons, token counts, errors
 - Chat history per node
 - Tool call details
 - Full context chain (what was sent to the agent)

Log rotation: Files rotate at 2MB to prevent unbounded growth.

12 Build and Run

12.1 Build Scripts

Script	What It Does
<code>npm run compile</code>	Compiles extension TypeScript → <code>dist/</code>
<code>npm run build:webview</code>	Builds React UI with Vite → <code>webview-ui/dist/</code>
<code>npm run build</code>	Both of the above
<code>npm run package</code>	Creates <code>.vsix</code> file for distribution

12.2 Development Workflow

1. Open the Zygote repo in VS Code
2. Press F5 to launch the Extension Development Host
3. In the new VS Code window, run command **Zygote: Open**
4. The Zygote webview panel opens on the right
5. Make changes to `src/` → recompile → restart Extension Host
6. Make changes to `webview-ui/` → rebuild webview → reload webview

12.3 Extension Manifest (package.json)

Key configuration:

- `main:` `"dist/extension.js"` — compiled entry point
- `engines.vscode:` `"^1.100.0"` — minimum VS Code version
- `activationEvents:` `["onCommand:zygote.open"]` — lazy activation
- `contributes.commands` — registers 3 commands
- `contributes.configuration.zygote.backend` — setting: `"auto" | "sdk" | "cli"`

13 What's NOT Built Yet (v0.1 Scope)

Understanding what Zygote *doesn't* do is just as important as what it does:

Missing Feature	Why It Matters
Branch merge	You can fork and switch, but not combine two branches
Shell/terminal execution	Agent can only read/write files, not run commands
Multi-turn agent loops	Each preview is a single agent turn (no iterative fix loops)
Side-by-side branch diff	No visual comparison of what changed between branches
Settings UI	Configuration is hard-coded
Cost tracking	No token usage dashboard or budget controls
Multi-user collaboration	Single-user only
Large workspace optimization	Designed for small-to-medium projects
Rejected preview history	Rejected previews are discarded (not saved for review)

14 Key Design Decisions Summary

Decision	Rationale	Trade-off
Flat node map	O(1) lookup, easy JSON serialization	Must traverse parentId for tree ops
Content-addressable snapshots	Dedup unchanged files across nodes	Must hash every file on capture
Extension host = source of truth	Webview sandbox can't do I/O	All state flows through one process
Auto-branching	Prevents accidental linear chains	May create unexpected branches
Two agent backends	Flexibility (SDK + CLI)	More code to maintain
Full tree sent on every update	Simple protocol, no partial diffs	Could be slow for very large trees
Immutable state ops	Predictable, testable, no side effects	Lots of object spreading

15 Quick Reference: File → Purpose Lookup

Use this table as a quick reference when navigating the codebase.

File	One-Line Purpose
src/extension.ts	Extension entry point, registers commands
src/webview/ZygotePanel.ts	Message routing hub, state orchestrator
src/webview/html.ts	HTML template for webview
src/state/tree.ts	Pure tree state operations
src/state/persistence.ts	Load/save tree.json
src/state/snapshots.ts	Content-addressable file versioning
src/agent/claude.ts	Anthropic API client
src/agent/tools.ts	4 tool definitions for the agent
src/agent/runner.ts	Agent execution + preview workflow
src/debug/logger.ts	Debug logging + tree dumps
src/shared/types.ts	TypeScript types for extension ↔ webview
webview-ui/src/App.tsx	Main React component + all UI state
webview-ui/src/vscode-api.ts	VS Code API wrapper for webview
webview-ui/.../TreeCanvas.tsx	Interactive tree visualization
webview-ui/.../NodeDetail.tsx	Right panel node details
webview-ui/.../NodeChat.tsx	Chat interface in draft nodes
webview-ui/.../FloatingChat.tsx	Quick-ask ephemeral popup
webview-ui/.../BranchSelector.tsx	Branch switching dropdown
webview-ui/.../MarkdownRenderer.tsx	Markdown + syntax highlighting
webview-ui/.../ThinkingBlock.tsx	Expandable thinking blocks
.zygote/tree.json	Persisted tree state (runtime)
.zygote/snapshots/*	File content blobs (runtime)
package.json	Extension manifest + dependencies
SPEC.md	Product specification

End of Zygote Architecture Guide

Read each section in order, or jump to any section by topic.
Use this document alongside the codebase to build your understanding.